
Applied Project 3: Imitation Learning

Group: Emilia Roma

Federica Morini
Giulia Pascoletti
Mattia Prandi

Abstract

We compare three imitation learning algorithms on two continuous-control Gym environments, Pendulum-v1 and MountainCarContinuous-v0. For each environment, we first train a PPO expert policy and use it to collect a master dataset of 100 expert trajectories. We then train IQ-Learn [Garg et al., 2021], f-IRL [Ni et al., 2020], and a SOAR-enhanced version of f-IRL [Viel et al., 2025] on subsets of size $K \in \{5, 20, 100\}$, averaging results over 3 random seeds. All methods are evaluated under a shared environment-interaction budget and compared against both expert and random-policy baselines.

Our results show that IQ-Learn is the most robust method across environments and dataset sizes. On Pendulum-v1, its performance improves consistently as the number of demonstrations increases, while f-IRL and f-IRL+SOAR reach high returns quickly but suffer from instability and late-stage collapse. This suggests that, in this environment, the main limitation is not exploration but the stability of the learned reward signal. On MountainCarContinuous-v0, all methods can reach near-expert performance, with f-IRL-based methods converging more slowly but remaining stable once the goal-reaching behavior is discovered. SOAR provides only limited improvements over standard f-IRL, indicating that it offers little benefit when the base method already explores successfully. Finally, visualizations of learned value and reward functions highlight a central property of IRL: successful imitation does not require recovering the true environment reward, since different learned rewards can induce similar expert-like policies.

1 Introduction

Reinforcement learning (RL) has achieved remarkable results on many environments but typically requires (i) a large number of interactions with the environment and (ii) a carefully designed reward function. In tasks where rewards are hard to specify by hand, or where exploration is expensive or unsafe, an attractive alternative is *imitation learning* (IL): the agent learns directly from expert demonstrations.

In this work we evaluate three representative IL algorithms in a controlled setting: ¹ IQ-Learn [Garg et al., 2021], which directly learns a soft Q-function from expert and online transitions, without explicitly modeling the reward function; *f*-IRL [Ni et al., 2020], which performs state-distribution matching between the expert and the learner via *f*-divergence minimization; and a SOAR-enhanced version of *f*-IRL [Viel et al., 2025], which replaces the single SAC critic pair with an ensemble and uses the disagreement among critics as an optimistic exploration bonus.

We are interested in three questions: **(Q1)** how does each algorithm scale with the number of expert demonstrations K , **(Q2)** does SOAR’s optimistic exploration help in regimes where the base *f*-IRL

¹Code available at <https://github.com/mattia-prandi/Reinforcement-Learning-Final-Project>

fails, and (Q3) in which settings is each algorithm the preferred choice? We answer all three questions empirically on two low-dimensional continuous control tasks. The choice of low-dimensional environments has two motivations: it keeps the compute budget tractable on a single laptop GPU and, more importantly, it allows us to visualize the learned reward function directly as a 2D heat map.

2 Background

RL studies how an agent learns to solve a task by interacting with an environment. At each timestep, the agent observes a state, selects an action, and receives a reward. The goal is to learn a policy that maximizes cumulative reward over time. Although RL has achieved strong results, it often requires many environment interactions and depends on carefully designed reward functions, which can be difficult to specify in complex or sparse-reward tasks.

Imitation Learning (IL) addresses these limitations by learning from expert demonstrations rather than relying only on trial-and-error exploration [Ng and Russell, 2000]. The agent observes trajectories generated by an expert policy and tries to reproduce similar behavior, which is useful when demonstrations are available or when exploration is expensive or unsafe. IL methods include Behavioral Cloning (BC), Interactive Imitation Learning, and Inverse Reinforcement Learning (IRL) [Ng and Russell, 2000]. In this project, we mainly focus on IRL-based methods, which aim to infer the value or reward structure underlying expert behavior rather than directly copying expert actions.

BC is the simplest IL method and treats imitation as supervised learning over expert state-action pairs [Pomerleau, 1988]. While efficient, BC suffers from cascading errors: small mistakes can move the agent into states not covered by the expert dataset, where the learned policy may fail. Interactive methods such as DAgger mitigate this issue by adding states visited by the learner to the training dataset [Ross et al., 2011], but require repeated interaction with the expert.

IRL provides a more flexible alternative by inferring the underlying objective that explains expert behavior, often as a reward function [Ng and Russell, 2000]. This can generalize since different agents may achieve the same goal through different actions or dynamics. Classical IRL approaches include feature-matching methods [Abbeel and Ng, 2004] and adversarial methods such as GAIL [Ho and Ermon, 2016].

Here, we focus on recent IL algorithms trained from fixed expert datasets, while still allowing the agent to interact with the environment to collect its own experience. We compare three methods. IQ-Learn directly optimizes a soft Q-function from which an implicit reward can be derived, avoiding a separate reward-learning stage [Garg et al., 2021]. f-IRL minimizes a divergence between the occupancy measures of the learner and expert policies, connecting IL to distribution matching [Ni et al., 2020]. Finally, SOAR introduces optimistic actor-critic updates to improve exploration and stability when learning from limited expert data [Viel et al., 2025].

The next section describes the experimental pipeline used to generate demonstrations, train the IL agents, and evaluate their performance.

3 Approach

To compare the performance of the considered IL algorithms, we adopt the following experimental pipeline. First, for each environment, we train an expert policy with PPO. Then, we use this expert to generate a fixed set of demonstration trajectories. Next, we construct expert datasets of different sizes by taking subsets of these trajectories. Finally, we train each IL method on the same expert datasets and evaluate the resulting policies in the original Gym environments. Since all algorithms are trained and evaluated under the same conditions, differences in performance can be largely attributed to the specific IL method and to the amount of expert data available.

3.1 Environments

We evaluate all methods on two continuous-control environments from the OpenAI Gym benchmark suite Brockman et al. [2016].

Pendulum-v1. A rigid pole attached to a fixed pivot must be balanced in the upright position. The state is $s = (\cos \theta, \sin \theta, \dot{\theta}) \in \mathbb{R}^3$, where θ is the angle from vertical, and the action is a scalar torque

$u \in [-2, 2]$ applied at the pivot. The reward is dense $r(s, u) = -(\theta^2 + 0.1 \dot{\theta}^2 + 0.001 u^2)$, so the agent is penalised for deviation from upright, for angular velocity, and for excessive control effort. The pole starts hanging downward, so the episode return is always negative; it approaches 0 only in the ideal limit and reaches approximately -1600 in the worst case.

MountainCarContinuous-v0. An underpowered car sits in a valley and must reach a flag on the right hill. The state is $s = (x, \dot{x}) \in \mathbb{R}^2$, where x is the horizontal position and $\dot{x}$ is the velocity, and the action is a continuous force $u \in [-1, 1]$ applied tangent to the slope. Because the engine is too weak to climb directly, the agent must first swing left to build momentum. The reward is sparse: the agent receives $+100$ upon reaching the flag and a small action penalty $-0.1 u^2$ at every step otherwise.

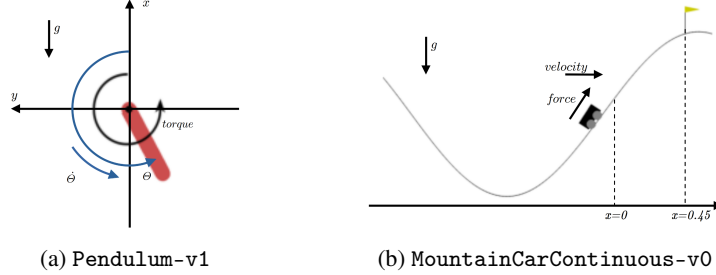


Figure 1: Schematic of the two evaluation environments.

Both environments feature continuous action spaces, which is a requirement of the SAC-based backbone shared by all three algorithms. The two environments are also complementary in difficulty: Pendulum provides a dense reward signal that is straightforward to imitate, while MountainCarContinuous presents a hard exploration problem.

3.2 Expert policy and demonstration dataset

For each environment, we train an expert policy π_E using Proximal Policy Optimization (PPO Schulman et al. [2017]), a stable on-policy actor-critic algorithm that reliably reaches near-optimal performance on low-dimensional continuous-control tasks within a modest step budget. Both experts were trained with the Stable-Baselines3 Raffin et al. [2021a] implementation. On MountainCarContinuous-v0 we enable generalised State-Dependent Exploration (gSDE Raffin et al. [2021b]), which produces temporally correlated actions that allow the car to build enough momentum to reach the goal.

After training, we roll out π_E deterministically and collect a master dataset of $K_{\max} = 100$ expert trajectories:

$$\mathcal{D}_E = \{\tau_i\}_{i=1}^{K_{\max}}, \quad \tau_i = \{(s_t, a_t, s_{t+1}, r_t, d_t)\}_{t=0}^{T_i-1}.$$

The rewards are stored only for evaluation and are never used during training of IL algorithms. To study the effect of demonstration quantity, we construct subsets of size $K \in \{5, 20, 100\}$ and train each IL algorithm on the same subsets.

3.3 IQ-Learn

IQ-Learn is one of the IL algorithms evaluated in this project. IQ-Learn formulates IL as an inverse soft Q-learning problem, learning a soft Q-function directly from expert demonstrations and online experience. Unlike classical IRL methods, which first recover a reward function and then optimize a policy, IQ-Learn combines both stages into a single optimization procedure in which the learned Q-function simultaneously guides the policy and induces an implicit reward.

In our implementation, the agent uses a SAC architecture with a stochastic Gaussian policy and a double-Q critic. Actions are squashed through a $\tanh$ function to satisfy the environment action bounds, while the double-Q architecture helps reduce value overestimation. At each training step, one batch is sampled from the replay buffer and one from the expert dataset. Expert transitions

contribute to the imitation objective, while both expert and online transitions are used to enforce Bellman consistency and learn a coherent soft Q-function.

IQ-Learn derives an implicit reward signal from the learned Q-function: $\hat{r}_Q(s, a, s') = Q(s, a) - \gamma V(s')$, where $Q(s, a)$ is the estimated action value and $V(s')$ is the soft value of the next state. Current Q-values are estimated using the main critics, while future soft values are computed using slowly updated target critics. This stabilizes the Bellman updates by providing a smoother estimate of future returns.

The actor is optimized by minimizing $J_\pi = \mathbb{E}[\alpha \log \pi(a|s) - Q(s, a)]$.

Intuitively, the actor is encouraged to select actions with high predicted value while maintaining sufficient exploration. During training, the true environment reward is never used and serves only as an evaluation metric.

Some modifications are introduced with respect to the standard SAC implementation to improve training stability. First, the entropy temperature is kept fixed at $\alpha = 10^{-2}$ instead of being automatically tuned. This choice is motivated by instabilities observed when combining entropy auto-tuning with tanh-squashed actions. Second, the χ^2 regularization term is applied only to expert transitions, which empirically results in more stable critic learning. Finally, gradient clipping is removed from critic updates, allowing the regularization term to react more strongly when Q-values start to diverge.

IQ-Learn is trained for 150 000 environment steps on Pendulum-v1 and 50 000 steps on MountainCarContinuous-v0. The replay buffer capacity is 100 000, and evaluations are performed every 5 000 steps over 10 deterministic episodes.

3.4 f-IRL

The second IL method evaluated in this project is f-IRL, which aims to match the state visitation distribution induced by the learned policy to the one induced by the expert. Therefore the goal is to make the learner generate trajectories that are similar to expert trajectories.

Let $\rho_E(s)$ be the expert state distribution and let $\rho_\pi(s)$ be the state distribution induced by the current policy π . f-IRL formulates IL as a distribution-matching problem and minimizes an f -divergence between these two state distributions. In our experiments, we use the reverse-KL variant:

$$\min_{\pi} D_{\text{KL}}(\rho_\pi \| \rho_E) = \min_{\pi} \int \rho_\pi(s) \log \frac{\rho_\pi(s)}{\rho_E(s)} ds.,$$

This objective penalizes the learned policy when it visits states that are likely under ρ_π but unlikely under the expert distribution ρ_E .

In our implementation, the f-IRL agent consists of three main components: a state discriminator, a learned reward model, and a Soft Actor-Critic (SAC) policy. The reward model $r_\psi(s)$ receives only the state as input and outputs a scalar learned reward. This reward is then used by SAC instead of the true environment reward during training. The discriminator $D_\omega(s)$ is a binary classifier trained to distinguish expert states, with label 1, from states generated by the current policy, with label 0.

Training alternates between policy optimization and reward learning. In our full experimental sweep, f-IRL is trained for 150 outer iterations on Pendulum-v1 and 50 outer iterations on MountainCarContinuous-v0. Each outer iteration corresponds to one complete f-IRL update cycle. First, SAC interacts with the environment for 1,000 steps using the current learned reward $r_\psi(s)$. During this phase, the actor and critic are updated from replay-buffer samples, while the reward model is kept fixed. The updated policy is then rolled out to collect 10 stochastic trajectories, whose states are samples from the current learner state distribution ρ_π . These on-policy states are compared with expert states to update the discriminator for 20 gradient steps. The reward model is then updated for 5 gradient steps using the f-IRL reverse-KL gradient estimator. For a policy trajectory $\tau = (s_0, \dots, s_T)$, the reward loss used in our implementation is proportional to the covariance

$$\mathcal{L}_r = \text{Cov}_{\tau \sim \pi} \left[\sum_t \log \frac{\rho_\pi(s_t)}{\rho_E(s_t)}, \sum_t r_\psi(s_t) \right].$$

As in IQ-Learn, the true environment reward is used only for evaluation. Every 10 outer iterations, the policy is evaluated over 10 deterministic episodes using the true environment reward. The actor and

critic are optimized with Adam at 3×10^{-4} , the reward network at 1×10^{-4} , and the discriminator at 3×10^{-4} .

3.5 SOAR-enhanced f-IRL

The third method is a SOAR-enhanced version of f-IRL, which modifies the SAC policy optimizer in order to improve exploration. The rest of the f-IRL pipeline remains unchanged.

The main difference is in the critic architecture used by SAC. In the base f-IRL implementation, SAC uses one double-Q critic to estimate the value of state-action pairs. The double-Q estimate is computed conservatively as $Q_{\text{SAC}}(s, a) = \min(Q_1(s, a), Q_2(s, a))$. In the SOAR-enhanced version, this single double-Q critic is replaced with an ensemble of $N = 4$ double-Q critic pairs. For a given state-action pair (s, a) , each ensemble member i produces two Q-value estimates, $Q_{i,1}(s, a)$ and $Q_{i,2}(s, a)$. As in standard SAC, the double-Q estimate is computed conservatively as $Q_{\text{SAC}}(s, a) = \min(Q_1(s, a), Q_2(s, a))$. This gives 4 Q-value estimates for the same state-action pair. Their mean represents the estimated value of the action, while the standard deviation measures critic disagreement, namely uncertainty. This disagreement is used to define an optimistic Q-value:

$$Q_{\text{SOAR}}(s, a) = \bar{Q}(s, a) + \beta \text{Std}_Q(s, a).$$

The standard deviation term acts as an exploration bonus, and the coefficient β controls the strength of this bonus. In our implementation, we use $\beta = 1$. All learning rates match those of the base f-IRL configuration described in Section ???. The actor is then optimized with the modified SAC objective:

$$\mathcal{L}_{\pi}^{\text{SOAR}} = \mathbb{E}_{s \sim \mathcal{B}, a \sim \pi} [\alpha \log \pi(a|s) - Q_{\text{SOAR}}(s, a)].$$

Since the actor minimizes this loss, it is encouraged to choose actions with high optimistic Q-values while still maintaining entropy through the $\alpha \log \pi(a|s)$ term.

We expect this modification to be most useful in environments where exploration is difficult. In particular, MountainCarContinuous-v0 requires the agent to first move back and forth to gain enough speed before reaching the goal. The uncertainty bonus introduced by SOAR may help the policy explore such trajectories, especially when the number of expert demonstrations is small.

3.6 Evaluation protocol

For each environment, dataset size, and algorithm, we train agents using three random seeds (seed $\in \{0, 1, 2\}$). The final comparison is based on the average undiscounted return obtained by the learned policy in the true Gym environment. All reported curves and summary statistics are averaged across seeds. We also include the PPO expert and a uniformly-random policy as reference baselines in the plots. The expert line indicates the target performance that IL should approach, while the random baseline provides a lower reference for task difficulty.

All experiments were run on a single laptop GPU (NVIDIA RTX 500 Ada, 4 GB VRAM). The full sweep of 54 runs took approximately 15–18 hours.

4 Results

The full experimental grid contains 54 runs (3 algorithms $\times$ 2 environments $\times$ 3 values of $K \times 3$ seeds). The following sections present the results together with their analysis and interpretation.

4.1 Learning Curves

Figure 3 reports the learning curves obtained for all considered algorithms, environments, and expert dataset sizes. The horizontal axis represents the number of environment interactions collected during training, while the vertical axis reports the average return obtained during evaluation. Each curve corresponds to the mean performance over three random seeds, and the shaded region represents one standard deviation. Higher returns indicate better imitation performance, with the dashed horizontal lines showing the performance of the PPO expert and a random policy, respectively.

The figure allows us to analyze how the amount of expert data affects learning, how quickly each algorithm approaches expert-level performance, and whether the learned policies remain stable during training.

Pendulum-v1

IQ-Learn IQ-Learn benefits significantly from increasing dataset size. With $K = 5$, the algorithm remains substantially below expert performance, reaching a best average reward of approximately -560 with high variance across seeds. With very few expert demonstrations, the critic is trained from a limited set of expert reference points. As a result, the implicit reward signal becomes noisier and the learned Q-function struggles to generalize across unseen regions of the state-action space, leading to poorer imitation performance. Increasing the number of demonstrations to $K = 20$ and $K = 100$ leads to a marked improvement in both performance and stability, with peak rewards around -240 and -218 , respectively, approaching the PPO expert reward. A mild performance degradation is observed for ($K=20$) during the later stages of training. Although the policy has already learned the task, the critic continues to be updated using bootstrapped value estimates. In the absence of a true reward signal that could anchor the learning process, small estimation errors may accumulate over time, leading to gradual Q-function drift and a corresponding decline in policy performance. The $K = 100$ configuration remains stable throughout most of the training horizon.

f -IRL f -IRL reaches near-expert performance much faster than IQ-Learn, even for small values of K . However, this is accompanied by substantial instability. For all dataset sizes, the learning curves exhibit a characteristic spike-and-collapse behavior: the policy briefly approaches expert-level performance and then rapidly degrades. A possible explanation is related to the pipeline linking discriminator and reward. The discriminator provides the signal that is used in the covariance-based reward update. Although the number of discriminator updates was reduced from 500 to 20 per outer iteration, Pendulum-v1 is a compact and low-dimensional environment and the discriminator can still overfit and produce saturated outputs. Therefore the estimated signal becomes less informative, the covariance gradient used to train the reward network becomes noisy, and SAC may optimize a corrupted learned reward. These results also suggest that, for Pendulum-v1, a shorter training horizon or an early-stopping criterion based on evaluation performance could preserve the high-performance phase before collapse. In this project, however, we keep a fixed budget of 150,000 environment steps to ensure a fair comparison across algorithms and dataset sizes.

f -IRL + SOAR The behavior of f -IRL + SOAR is qualitatively similar to that of standard f -IRL. The algorithm quickly reaches high reward, but it also suffers from large fluctuations and late-stage degradation. This indicates that the optimistic exploration mechanism introduced by SOAR does not remove the main instability in this environment. The reason is that SOAR only modifies the SAC critic and actor update by adding an exploration bonus. In this setting, the bottleneck is therefore not insufficient exploration, but the stability of the learned reward signal. For this reason, SOAR does not provide a substantial advantage over standard f -IRL on Pendulum-v1.

MountainCarContinuous-v0

IQ-Learn. IQ-Learn rapidly reaches near-expert performance for all dataset sizes. Even with only $K = 5$ trajectories, the algorithm achieves rewards close to 95. However, a late-training performance degradation becomes visible and grows with dataset size. For $K = 5$, the effect is relatively mild, with a final reward of approximately 89.8. For $K = 20$, the policy oscillates between 63 and 95 during the final 20 000 environment steps, while for $K = 100$ a sharp drop occurs at the last evaluation, resulting in a final reward of approximately 68.9. A possible explanation is related to the increasing number of terminal transitions observed once the task is consistently solved. In MountainCarContinuous-v0, episodes terminate when the goal is reached, so a successful policy rapidly fills the replay buffer with terminal transitions. For such transitions, the soft value term vanishes ($(V(s')=0)$), causing the implicit reward estimate to depend directly on the Q-value. Consequently, the critic becomes more sensitive to inaccuracies in its own value estimates. Since the Bellman objective and the chi-squared regularization both act on the learned value function, inaccuracies may accumulate over time and lead to less stable critic updates.

f -IRL. f -IRL converges more slowly than IQ-Learn but eventually reaches near-expert performance for all values of K , achieving rewards around 94. Larger datasets mainly improve convergence speed rather than final performance. Once the policy discovers the goal-reaching behavior, training remains stable and shows very low variance across seeds. Compared to Pendulum-v1, the discriminator

appears much more reliable in this environment. The reduced number of discriminator updates (20 per outer iteration) is sufficient to keep the discriminator signal informative without causing the saturation observed on Pendulum. As a result, the learned reward remains useful for SAC throughout training.

f -IRL + SOAR. f -IRL + SOAR displays behavior very similar to standard f -IRL. The benefits of optimistic exploration are limited, with only minor differences in convergence speed compared to the baseline. Once the policy consistently reaches the goal, the performance of the two methods becomes nearly indistinguishable. A possible explanation is that, after the goal-reaching behavior has been learned, the ensemble critics become increasingly consistent with each other. Consequently, the standard deviation term used by SOAR becomes small, and the optimistic exploration bonus has little effect. Since standard f -IRL already learns a sufficiently good reward and reaches near-expert performance on this environment, the additional exploration mechanism does not provide a clear measurable benefit.

4.2 Visualization of Learned Reward and Value Functions

Figure 4 visualizes the functions learned by the different IL algorithms. For IQ-Learn, the heatmaps show the learned soft value function $V(s)$, since the method does not explicitly learn a reward network. For f -IRL and f -IRL + SOAR, the heatmaps show the learned reward function $\hat{r}(s)$ produced by the reward network. To facilitate interpretation, the ground-truth environment reward is also shown in figures 4a and 4c for both environments, although no corresponding ground-truth value function is available.

The horizontal and vertical axes correspond to the state variables of each environment, namely the angle θ and angular velocity $\dot{\theta}$ for Pendulum-v1, and the car position and velocity for MountainCarContinuous-v0. The white points represent states visited by the expert demonstrations, providing a visualization of the expert state distribution. Colors indicate the magnitude of the plotted function: brighter and darker regions corresponds respectively to higher and lower reward or value estimates.

For IQ-Learn, the value function is computed using the best-performing critic checkpoint together with the corresponding actor policy. For f -IRL and f -IRL + SOAR, the heatmaps are obtained from the best-performing reward network and its associated policy checkpoint. The selected checkpoints correspond to the highest evaluation performance achieved during training.

Several observations can be drawn from these visualizations.

First, the heatmaps learned by IQ-Learn differ substantially from those of f -IRL and f -IRL + SOAR, since the plotted quantities are fundamentally different. IQ-Learn learns a soft value function $V(s)$ so it tends to assign high values not only to goal states, but also to states from which the goal can be reached easily in the future.

For Pendulum-v1, both f -IRL and f -IRL + SOAR correctly assign the highest rewards to states near the upright equilibrium, corresponding to the maximum of the ground-truth reward. Although the learned rewards do not perfectly match the true reward landscape, they capture the most important structure of the task: states close to the goal receive higher rewards than states far from it. The value functions learned by IQ-Learn exhibit a different structure, assigning high values to a broader region of the state space. This behavior is consistent with the interpretation of $V(s)$ as a measure of future expected return rather than immediate reward.

For MountainCarContinuous-v0, some learned reward functions assign high values near the goal region, whereas others place high rewards in a different part of the state space. Despite these differences, the resulting policies achieve very similar performance. This observation illustrates a fundamental property of inverse reinforcement learning: the reward function is generally not uniquely identifiable from demonstrations alone. Multiple reward functions can induce the same optimal behavior. In particular, successful policies in MountainCarContinuous-v0 must first move away from the goal and build momentum by driving up the left hill before reaching the goal on the right. Consequently, a reward function that strongly values states on the left side of the environment can still induce the correct behavior and achieve near-expert performance.

5 Conclusion

In this work, we compared IQ-Learn, (f)-IRL, and (f)-IRL + SOAR on two continuous-control environments while varying the number of expert demonstrations.

Regarding **Q1**, we observe that IQ-Learn scales most consistently with the amount of expert data. On Pendulum-v1, performance improves steadily as (K) increases, while on MountainCarContinuous-v0 the algorithm reaches near-expert performance even with only a few demonstrations. In contrast, (f)-IRL and (f)-IRL + SOAR are considerably more sensitive to the environment. While both methods perform well on MountainCarContinuous-v0, they exhibit significant instability on Pendulum-v1, where increasing (K) does not necessarily lead to improved performance.

Regarding **Q2**, our experiments indicate that SOAR provides only limited benefits over standard (f)-IRL. The dominant limitations on Pendulum-v1 appear to originate from reward learning rather than insufficient exploration. On MountainCarContinuous-v0 both methods already reach near-expert performance, leaving little room for improvement.

Finally, regarding **Q3**, IQ-Learn emerges as the most robust method overall. It consistently benefits from additional demonstrations, produces stable policies, and most closely matches the expert state distribution, as illustrated by the state visitation analysis figure 2.

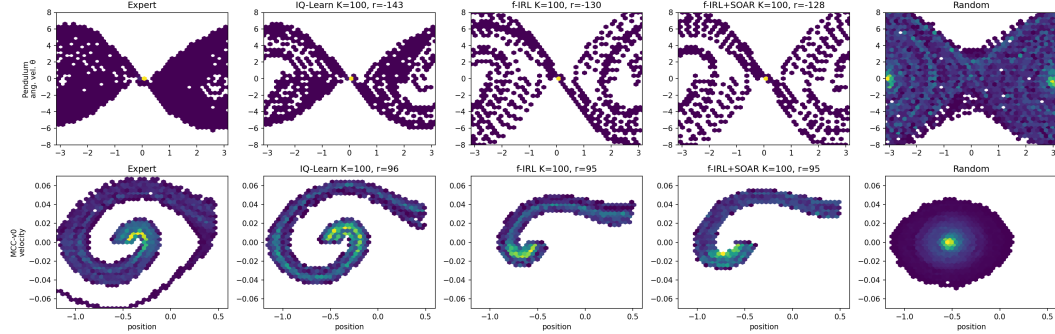


Figure 2: State visitation densities induced by the expert policy, the best policy learned by each IL algorithm ($K = 100$), and a random policy. Each point represents a region of the state space, and its color indicates how frequently that region is visited during policy execution. The top row corresponds to Pendulum-v1, the bottom row corresponds to MountainCarContinuous-v0.

The reward and value visualizations figure 4 highlight a key property of IL. Successful imitation does not require recovering the true environment reward, multiple reward functions may explain the same expert behavior. In MountainCarContinuous-v0, substantially different learned reward functions produce policies with nearly identical performance.

References

- Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the Twenty-First International Conference on Machine Learning (ICML)*, 2004.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- Divyansh Garg, Shuang Ma, Jian Zhang, Ruslan Salakhutdinov, and Aviral Kumar. Iq-learn: Inverse soft-q learning for imitation. In *Advances in Neural Information Processing Systems 34*, 2021.
- Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems 29*, 2016.
- Andrew Y. Ng and Stuart Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML)*, pages 663–670, 2000.
- Tianwei Ni, Harshit Sikchi, Yufei Wang, Tejus Gupta, Lisa Lee, and Benjamin Eysenbach. f-irl: Inverse reinforcement learning via state marginal matching. In *Proceedings of the 4th Conference on Robot Learning (CoRL)*, 2020.
- Dean A. Pomerleau. Alvin: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems 1*, 1988.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021a.
- Antonin Raffin, Jens Kober, and Freek Stulp. Smooth exploration for robotic reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, 2021b.
- Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Stefano Viel, Luca Viano, and Volkan Cevher. Il-soar: Imitation learning with soft optimistic actor critic. *arXiv preprint arXiv:2502.19859*, 2025.

AI Usage Declaration

We used LLM assistants in the following capacities.

Report and poster writing. We drafted the content ourselves and used an LLM to convert our notes into LaTeX, fix grammatical errors, improve the fluency of the text, and condense paragraphs to meet the report length requirements. The scientific content, arguments, and conclusions are entirely our own.

Code generation. We used an LLM to generate secondary code such as plotting scripts and to autocomplete implementation details. Every generated code block was reviewed line by line and verified against the pseudocode in the original papers before being accepted.

Concept clarification. After discussing unclear concepts among ourselves, we used an LLM to clarify our understanding. The LLM was consulted only after an initial effort to resolve the question independently.

Validation of hypotheses and conclusions. We used an LLM to cross-check our interpretations and conclusions. Whenever the LLM disagreed with our analysis, we investigated whether it was hallucinating or correct; in the latter case, we revisited the relevant material until we reached a full understanding before updating our conclusions.

Contribution Statement

The project was carried out collaboratively, with all group members participating in the main design decisions, implementation choices, experimental setup, and discussion of results.

To enable a deeper understanding of the different methods, each team member took primary responsibility for studying and implementing a specific component of the project:

- **Mattia Prandi** focused on the environments and the PPO expert training pipeline. He also implemented a substantial portion of the code.
- **Federica Morini** focused on the implementation and analysis of IQ-Learn and reviewed the code.
- **Giulia Pascoletti** focused on the implementation and analysis of f -IRL and f -IRL + SOAR and reviewed the code.

This division of responsibilities proved particularly effective, as each member subsequently shared their findings with the rest of the team, allowing all group members to develop a solid understanding of the complete project while accelerating the learning process.

The interpretation of the experimental results, the analysis of the learned policies and reward functions, and the writing of the final conclusions were carried out jointly by all team members.

A Figures

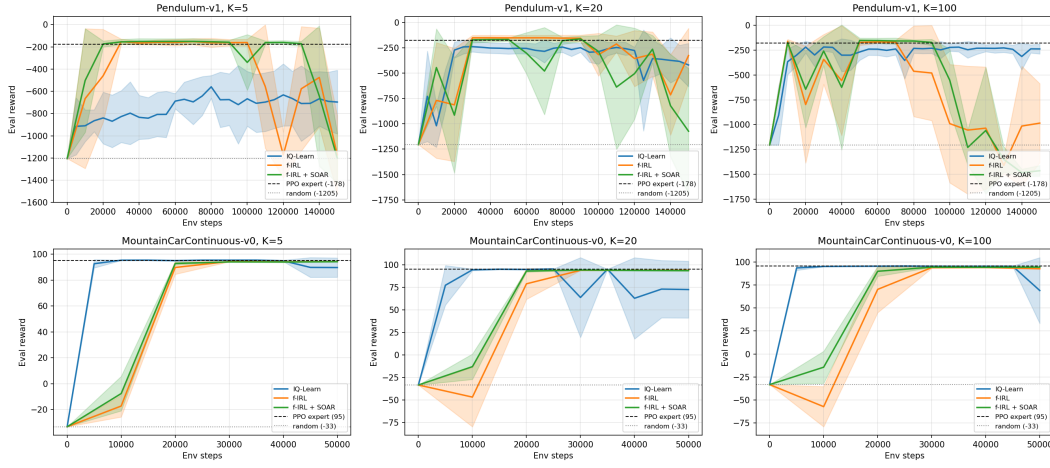


Figure 3: Learning curves of IQ-Learn, f -IRL, and f -IRL + SOAR on Pendulum-v1 and MountainCarContinuous-v0 for different expert dataset sizes K .

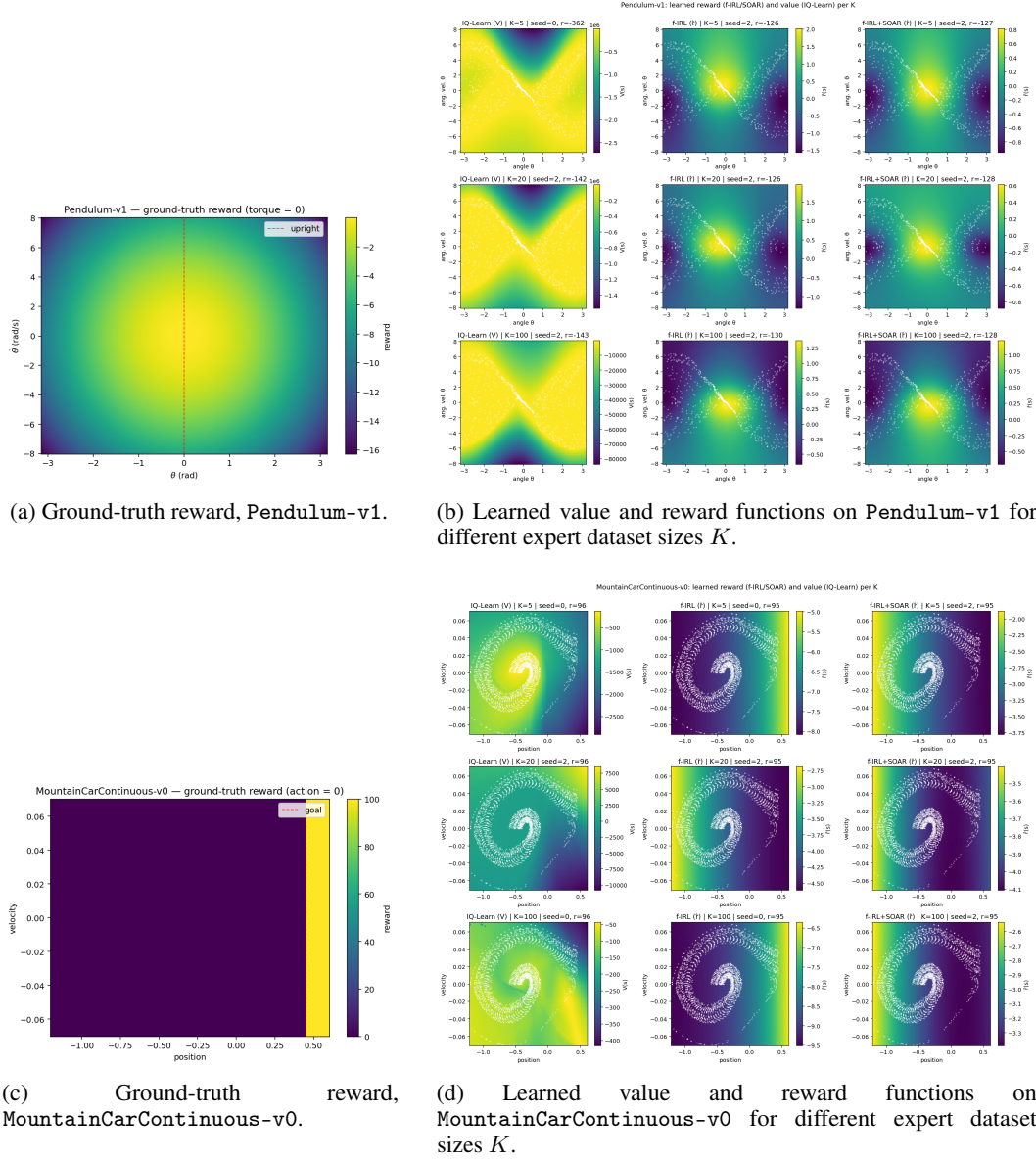


Figure 4: Heatmaps representing the ground-truth reward and the learned value/reward functions for the considered IL algorithms. White points indicate states visited by the expert demonstrations.